



Russound Dual Tuners **(ST2, ST2-XM, ST2-XM2, and ST2S)**

RNET™ Protocol & Specifications **RS-232 Communication**

Document version 1.00.00

NOTE: This document is intended for ST2 Chassis Firmware Version 2.00.06 or higher. The XM and Sirius Firmware must be V1.00.10 or higher.

Table of Contents:

1	INTRODUCTION.....	3
2	OVERVIEW	3
3	COM PORT.....	3
3.1	COM SETTINGS.....	3
3.2	CONNECTOR TYPE / PIN-OUT.....	3
4	RNET™ MESSAGE PACKET FORMAT	4
4.1	MESSAGE HEADER	4
4.1.1	<i>Start of Message Character</i>	<i>4</i>
4.1.2	<i>Target Device ID</i>	<i>4</i>
4.1.2.1	Target Controller ID	4
4.1.2.2	Target Zone (Port) ID	4
4.1.2.3	Target Keypad ID	5
4.1.2.4	Target Device ID Example	5
4.1.3	<i>Source Device ID.....</i>	<i>5</i>
4.1.3.1	Source Controller ID.....	5
4.1.3.2	Source Zone (Port) ID.....	6
4.1.3.3	Source Keypad ID.....	6
4.1.4	<i>Message Type.....</i>	<i>6</i>
4.1.4.1	Event Messages.....	6
4.1.5	<i>Paths</i>	<i>6</i>
4.2	THE INVERT CHARACTER	7
4.3	MESSAGE BODY	8
4.4	MESSAGE TRAILER.....	8
4.4.1	<i>Checksum.....</i>	<i>8</i>
4.4.2	<i>End of Message Character</i>	<i>8</i>
5	COMMON CONTROL MESSAGES (AM/FM, XM, AND SIRIUS)	9
5.1	POWER ON / POWER OFF	9
5.2	TUNE UP / TUNE DOWN (FREQUENCY OR CHANNEL)	9
5.3	DIRECT TUNING MODE (FREQUENCY OR CHANNEL)	9
5.4	SAVING PRESETS	10

5.5	PRESET UP / PRESET DOWN	11
5.6	DIRECT PRESET SELECT	11
5.7	BANK UP / BANK DOWN	11
5.8	DIRECT BANK SELECT	11
6	AM/FM TUNER CONTROL MESSAGES	13
6.1	SEEK	13
6.2	SCAN	13
6.3	AM/FM TOGGLE	13
6.4	DIRECT AM SELECT / DIRECT FM SELECT	13
6.5	STEREO / MONO TOGGLE	14
6.6	DIRECT STEREO SELECT / DIRECT MONO SELECT	14
6.7	LOCAL / DISTANT TOGGLE	14
6.8	DIRECT LOCAL SELECT / DIRECT DISTANT SELECT	14
7	SATELLITE RADIO TUNER CONTROL MESSAGES (XM AND SIRIUS)	15
7.1	CATEGORY UP / CATEGORY DOWN	15
7.2	DIRECT CATEGORY SELECT	15
7.3	CATEGORY CHANNEL UP / CATEGORY CHANNEL DOWN	16
8	DISPLAY MESSAGES	17
8.1	READING DIRECT DISPLAY FEEDBACK	17
8.2	READING SOURCE BROADCAST DISPLAY FEEDBACK	18
8.3	READING MULTI-FIELD BROADCAST DISPLAY FEEDBACK MESSAGES	18
9	USING THE ACKNOWLEDGE MESSAGE (HANDSHAKING)	20
10	COMMAND CODE SUMMARY FOR PERIPHERAL TUNER	21
11	ASCII CHARACTER SET TO HEX CONVERSION CHART	23

1 Introduction

This document describes how to control and interpret data from the Russound Dual Tuners through their RS-232 serial port using the following **RNET™** communications protocol. It is assumed that the reader is familiar with the features and operation of the ST2 and its variants. All commands use Hexadecimal or HEX values. In descriptive text these are indicated by preceding the value with a **0x**. Zero-based values are also used in certain areas and are noted as such. This means that a value of **0x00 = 1, 0x01 = 2, and 0x02 = 3** etc... Throughout this document all bytes within message packets not in bold must be referenced exactly as they appear.

2 Overview

The Russound Dual Tuner and its variants use a comprehensive Communications Protocol called **RNET™** which has been extended to the RS-232 port on its rear panel. Through this port, virtually all aspects of the device operation can be performed. This document provides detailed descriptions of messages required to perform basic device operation, as well as instructions on how to interpret Display Feedback messages.

3 COM Port

The following information is needed to allow connection and communication with the ST2 Chassis.

3.1 COM Settings

The following settings must be made to the RS-232 serial COM port used to interface with the ST2 and its variants:

- 19200 baud rate
- 1 Start bit
- 1 Stop bit
- No flow control
- No parity

3.2 Connector Type / Pin-out

Connector: Female DB-9

- Pin 1: NC
- Pin 2: Transmit
- Pin 3: Receive
- Pin 4: NC
- Pin 5: Ground
- Pin 6: NC
- Pin 7: NC
- Pin 8: NC
- Pin 9: NC

4 RNET™ Message Packet Format

Header				Body	Trailer	
0xF0	Target Device ID	Source Device ID	Type	Body	Checksum	0xF7

Every **RNET™** message has a consistent format. There are three major components: Message Header, Message Body and Message Trailer. Each of these is explained here in some detail.

MSB stands for **Most Significant Bit** which is the far left Bit of an 8-Bit Byte (MSB = Bit 7 = **00010001**). There are several special characters used in this protocol. These special characters have the MSB set to high (1). All Bytes within the message of an **RNET™** packet have the MSB set to low (0) except for three special characters. The special characters with the MSB set to high (1) are the “**Start of Message Character**”, the “**End of Message Character**”, and the “**Invert Character**”, which are explained within this document.

4.1 Message Header

Each message consists of a “Message Header” which is compiled of a **Start of Message Character**, **Target Device ID**, **Source Device ID**, and the **Message Type**.

4.1.1 Start of Message Character

As previously stated, there are several special characters. One of these is the Start of Message Character **0xF0** which is a special HEX value that indicates the beginning of a new message.

4.1.2 Target Device ID

The Target Device ID defines to which device the message is being sent. Every device on an **RNET™** system has a unique “Device ID” that allows messages to be sent to it. Each Device ID consists of a Controller ID, a Zone (Port) ID, and a Keypad ID. In the case of a Controller with zone Keypads, the purpose of each Device ID field is apparent. In the case of the Dual Tuner and all its variants, these fields will have different meanings because this is an **RNET™** Peripheral Device and not a Controller. These fields are explained in more detail below.

4.1.2.1 Target Controller ID

When communicating with a Dual Tuner and all its variants, the Controller ID is always set to a value of **0x00**. When the Dual Tuner is responding with a Display Feedback message, a value of **0x7D** will be seen in this field targeting the entire **RNET™** system.

4.1.2.2 Target Zone (Port) ID

When communicating with a Dual Tuner and all its variants, the Zone ID is always set to **0x7D**. The value **0x7D** in the Zone (Port) ID field indicates that the Target Device is an **RNET™** Peripheral. When the Dual Tuner is responding with a Source Broadcast Display Feedback message, a value of **0x00** will be seen in this field since the Target Controller ID already indicated All **RNET™** Devices, not a particular Zone of a Controller.

4.1.2.3 Target Keypad ID

The Keypad ID field is a zero-based value that is used to indicate the Source Number of each Tuner module within the Dual Tuner to be controlled. See the Dual Tuner operating instructions for how to set the Source Numbers for each Tuner module. The available Source Numbers are Source #1 through Source #6 and therefore use HEX values of **0x00** through **0x05**. The Source Numbers can also be set as “Unassigned” in which case you can not Target that Tuner with an **RNET™** protocol message. When receiving messages from the Dual Tuner, this field will use special values to indicate where the message is being sent. A value of **0x70** in this field would indicate that a Direct Display Feedback message is being sent from the Peripheral Device. A value of **0x79** in this field would indicate that a Broadcast Display Feedback message is being sent from the Peripheral Device.

4.1.2.4 Target Device ID Example

Example 1 shows a complete Target Device ID for a Dual Tuner that has one of its Tuner modules assigned as Source 1 in an **RNET™** system.

NOTE: The Dual Tuner has two tuner modules and therefore the two modules are viewed as two separate devices on the **RNET™** network. The Dual Tuner has two Device IDs that it will respond to.

Example 1:

Value (HEX)	Device ID Field	Description
00	Controller ID	ST2
7D	Zone (Port) ID	RNET™ Peripheral
00	Keypad ID	Source 1 = 0x00 (Zero-based, see 4.1.2.3)

Example 2 shows a complete Target Device ID for a Source Broadcast Display message being sent from a Dual Tuner in an **RNET™** system.

Example 2:

Value (HEX)	Device ID Field	Description
7D	Controller ID	All RNET™ Devices
00	Zone (Port) ID	Arbitrary value
79	Keypad ID	Broadcast Display Feedback message

4.1.3 Source Device ID

The “Source” Device ID is the identification of the Device that is sending the message. For external devices controlling the system, like an automation and control system, these can be any value that is a unique value among devices attached to the system. The recommend Device ID for external control systems is **Controller ID: 0x00**, **Zone ID: 0x00**, and **Keypad ID: (0x70 or 0x60)**.

4.1.3.1 Source Controller ID

For 3rd party devices, this should be set to a value of ‘**0x00**’.

4.1.3.2 Source Zone (Port) ID

For 3rd party devices, this should be set to a value of '**0x00**'. When the Dual Tuner is responding with a Display Feedback message, a value of **0x7D** will be seen in this field indicating a Peripheral Device.

4.1.3.3 Source Keypad ID

The recommend Keypad ID for external control systems is **0x70**, which is a Keypad ID other than those used in the system. If using **0x60**, then the system will require a Handshake message. When the Dual Tuner is responding with a Display Feedback message, this field will indicate the zero-based value for the Source Number (**0x00** – **0x05**) of the Tuner sending the message.

4.1.4 Message Type

This value defines the type of message that is being sent. The most important Message Types are as follows:

Value (HEX)	Message Type	Description
00	Set Data	Sets a parameter's value
01	Request Data	Requests a parameter's value
02	Handshake	Acknowledges a Data send
05	Event	Triggers a system response that may set a parameter value, update displays, etc...

In some cases, setting parameter values can be done in two ways. A **Set Data** message can be sent directly to the parameter, or an **Event** message can be sent to trigger the Dual Tuner to set the value instead. **Event** messages are a little easier to use, and may trigger other desired results (such as updating Keypad displays, updating related parameters, etc...) where a **Set Data** message may not. Because of this, this document describes using **Event** messages to set parameter values where it is most desirable.

4.1.4.1 Event Messages

Event Messages trigger a system response that may set a parameter value, update displays, etc... Because of this, this document describes using **Event Messages** to set parameter values where it is most desirable. An **Event Message** Type consists of an Event ID, Event Timestamp, Event Data, and the Event Priority.

4.1.5 Paths

The Data within each RNET device is organized in a hierarchical tree similar to a computer's disk drive. When sending Data or Events from one system to another, a Path is specified to define exactly which object should be modified. Target and Source Paths are usually defined, but for the sake of remotely controlling the system, only the Target Path is of interest.

Paths in RNET messages are defined by first specifying how deep the path is, followed by the numeric values of each directory. For example, a Path that was three levels deep might be represented as “**3.1.2.8**”. The “**3**” at the beginning defines how deep the Path is. The numbers that follow are the subdirectories. Think of the numbers as folder names (folder “**8**” is inside folder “**2**” which is inside folder “**1**”). Bear in mind that the folders are unique for each level, so folder “**1**” can be in folder “**1**” of the next level up (just like folders on a computer).

4.2 The Invert Character

The **Invert Character** is another one of the special characters. It is used in special cases as part of the Message Body. If the data in an **RNET™** message body includes any Byte values that have the MSB set to high (1) the Byte will be rejected as only the lower 7 Bits are used to hold data. When the MSB is set to high (1) the Byte has a HEX value greater than **0x7F**. In order to allow values greater than **0x7F** to be accepted, the Byte must first be Bitwise inverted (e.g., 10010101 = 01101010), and the special Invert Character (**0xF1**) is inserted just prior to the inverted Byte. When an **RNET™** packet is received, the system must detect the **0xF1** Invert Character. The **0xF1** Invert Character is then discarded and the following Byte is inverted back to its original value (e.g., 01101010 = 10010101).

Invert Character Usage Example:

Sent message with inverted character:

Value (HEX)	Notes
F0	Start of message character
00	
67	
7C	
F1	Special Invert Character
6A	Inverted Character (actual value 0x95)
34	Checksum
F7	End of message character

Received message after discarding the Invert Character and re-inverting the following Byte:

Value (HEX)	Notes
F0	Start of message character
00	
67	
7C	
95	After re-inverting
XX	Checksum (Discarded)
F7	End of message character

4.3 Message Body

The message body contains specific data which varies in value and byte count depending on the particular **Message Type** being sent. Refer to the particular messages.

4.4 Message Trailer

The Message Trailer consists of the **Checksum** and **End of Message Character**. The **Checksum** value changes and needs to be re-calculated whenever any one of the preceding characters in the message is changed. The **Checksum** is always followed by the **End of Message Character**.

4.4.1 Checksum

All messages include a Checksum that helps protect the integrity of the message. The Checksum is a single Byte that can be calculated using a formula (see example below). The **Checksum** itself and the **End of Message Character** are not included in the calculation. Only the low 7 Bits are used so overflow is discarded.

Checksum Calculation Example:

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	00	00	00	70	05	02	01	00	02	01	00	3A	00	70	00	00	00	01	28	F7

Step #1 – Add the HEX value of every Byte in the message that precedes the Checksum:

Example: $0xF0 + 0x00 + 0x7D + 0x00 + 0x00 + 0x00 + 0x70 + 0x05 + 0x02 + 0x01 + 0x00 + 0x02 + 0x01 + 0x00 + 0x3A + 0x00 + 0x70 + 0x00 + 0x00 + 0x00 + 0x01 = 0x0293 = 0x93$

Step #2 – Count the number of Bytes which precede the Checksum and convert that value from DEC to HEX (this is called the Byte Count):

Example: DEC Byte Count = **21** / Converted to HEX = **0x15**

Step #3 – Add the HEX Byte Count found in Step #2 to the sum of Bytes calculated in Step #1:

Example: $0x15$ (HEX Byte Count) + $0x93$ (sum of Bytes) = **0xA8**

Step #4 – The value from Step #3 is then **AND**-ed with $0x7F$ (the highest BIN value for 7 bits = $1111111 = 0x7F$):

Example: $0xA8 \text{ AND } 0x7F = 0x28 = \text{Checksum}$

4.4.2 End of Message Character

The third special character is the last Byte in the message called the **End of Message Character**. It is the special HEX value **0x07** that indicates the end of the message.

5 Common Control Messages (AM/FM, XM, and Sirius)

This section will provide information in regards to messages that apply to all three different ST2 Tuner Modules. The following strings are common command messages that would apply to the AM/FM Tuner as well as both Satellite Radio Tuners (XM and Sirius).

5.1 Power On / Power Off

This section describes how to initiate the Power On and Power Off functions. The Power On function wakes the Dual Tuner from “Standby Mode”. The Power Off function puts the Dual Tuner into “Standby Mode”.

NOTE: *The Power On and Power Off commands are common to both tuners. If a Power command is sent to one tuner it will affect both tuners (The entire ST2 Chassis). There is no need to address the second tuner for these functions.*

This is an example of a Power On / Power Off message

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Power On = **0x3A** / Power Off = **0x3B**)

5.2 Tune Up / Tune Down (Frequency or Channel)

The same HEX values for Tune Up and Tune Down are used for both Frequency Tuning and Channel Tuning. For AM/FM issuing this message will Tune Up or Tune Down one Frequency at a time. For XM and Sirius issuing this message will Tune Up or Tune Down one Channel at a time.

This is an example of Tune Up / Tune Down relative to the current Frequency or Channel

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Frequency / Channel Up = **0x2F**) (Frequency / Channel Down = **0x30**)

5.3 Direct Tuning Mode (Frequency or Channel)

Direct Tuning mode will allow you to directly select a particular Frequency for AM/FM (i.e. 101.7 FM) or a particular Channel for XM and Sirius (i.e. XM Channel 20). The AM/FM Tuner will automatically detect if the entered frequency is intended for AM or FM based on the number sequence. For all three Tuners (AM/FM, XM, and Sirius) the command sequence includes a series of messages. The first message is to tell the AM/FM or Satellite Radio Tuner to enter Direct Tuning mode. The proceeding messages are to indicate what Frequency or Channel Digits to select.

This is the message used to enter **Direct Tuning** mode (Frequency or Channel).

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	17	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Direct Tuning mode = **0x17**)

This is an example of the proceeding messages to select the **Digits** (Frequency or Channel). The maximum number of Digits for AM/FM would be four Digits. The maximum number of Digits for Satellite Radio would be three Digits.

Digit #1:

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

Digit #2:

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

Digit #3:

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

Digit #4:

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Digit #1 = **0x01**, #2 = **0x02** ... #9 = **0x09**) (**NOTE:** Digit #0 = **0x0A**)

5.4 Saving Presets

There are six HEX values that are used as a “Save” command for each of the six available Presets. When you send one of the Save Preset # commands it will be applied to the currently selected Bank.

This example shows how to save Presets (1-6) within the currently selected Bank for the currently selected Frequency / Channel using a discrete message

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Save Preset 1 = **0x3C**, 2 = **0x3D**, 3 = **0x3E**, 4 = **0x3F**, 5 = **0x40**, 6 = **0x41**)

5.5 Preset Up / Preset Down

This is an example of Preset Up / Preset Down within the current Bank

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Preset Up = **0x0E** / Preset Down = **0x0F**)

5.6 Direct Preset Select

Direct Preset Select mode allows you to select a particular Preset (1 – 6) within the currently selected Bank. Just like Direct Tuning mode, the command sequence includes a series of messages. The first message is to put the Tuner into Direct Preset Select mode. The second message is to indicate what Preset Digit to select.

This is the message used to enter **Direct Preset Select** mode

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	16	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Direct Preset Select mode = **0x16**)

This is an example of the second message needed to select the Preset **Digit** (1 – 6)

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Digit #1 = **0x01**, #2 = **0x02**, #3 = **0x03**, #4 = **0x04**, #5 = **0x05**, #6 = **0x06**)

5.7 Bank Up / Bank Down

This is an example of Bank Up / Bank Down relative to the current Bank

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Bank Up = **0x29** / Bank Down = **0x2A**)

5.8 Direct Bank Select

Direct Bank Select mode allows you to select a particular Bank (1 – 6). Again, the command sequence includes a series of messages. The first message tells the Tuner to enter Direct Bank Select mode. The second message is to indicate what Bank Digit to select.

This is the message used to enter **Direct Bank** Select mode.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	15	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Direct Bank Select mode = **0x15**)

This is an example of the second message needed to select the Bank **Digit** (1 – 6)

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	00	00	00	70	05	02	01	00	02	01	00	bb	00	70	00	00	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Digit #1 = **0x01**, #2 = **0x02**, #3 = **0x03**, #4 = **0x04**, #5 = **0x05**, #6 = **0x06**)

6 AM/FM Tuner Control Messages

This section provides information in regards to messages that **only** apply to the AM/FM Tuner module. These messages include Seek, Scan and AM/FM Band selection commands.

NOTE: Sending a “**Power On**” message will stop a Seek or Scan operation! Refer to [Section 5.1](#) for the message used as “Power On”.

6.1 Seek

The Seek function finds the next “Tuned Station” within the current Band.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	1B	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Seek = **0x1B**)

6.2 Scan

The Scan function finds the next Tuned Station within the current Band. It will rest on that Station for 5 seconds and then find the next Tuned Station, and so on. This sequence will continue until a “Power On” command is sent to stop the Scan function.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	46	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Scan = **0x46**)

6.3 AM/FM Toggle

The AM/FM Toggle function switches back and forth between AM and FM Bands.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	1A	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (AM/FM Toggle = **0x1A**)

6.4 Direct AM Select / Direct FM Select

Direct AM/FM Selection will directly select the AM or FM band using a discrete message.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Direct AM Select = **0x4A**) (Direct FM Select = **0x49**)

6.5 Stereo / Mono Toggle

The Stereo / Mono Toggle message switches back and forth between Stereo and Mono modes when listening to an FM Station.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	13	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Stereo / Mono Toggle = **0x13**)

6.6 Direct Stereo Select / Direct Mono Select

The Direct Stereo Select and Direct Mono Select functions discreetly select the Stereo Listening mode or Mono Listening mode when listening to an FM Station.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Direct Stereo Select = **0x33**) (Direct Mono Select = **0x34**)

6.7 Local / Distant Toggle

The Local / Distant Toggle message switches back and forth between Local Tuning mode and Distant Tuning mode.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	14	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Local / Distant Toggle = **0x14**)

6.8 Direct Local Select / Direct Distant Select

The Direct Local Select and Direct Distant Select functions discreetly select the Local Tuning mode or Distant Tuning mode.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Direct Local Select = **0x42**) (Direct Distant Select = **0x43**)

7 Satellite Radio Tuner Control Messages (XM and Sirius)

This section provides information in regards to messages that apply specifically to the XM Tuner and Sirius Tuner modules. This will include Category selection commands.

7.1 Category Up / Category Down

Category Up and Category Down will select the next or previous Category relative to the currently selected Category.

NOTE: Once a Category is selected, the first Channel of that Category will be selected.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Category Up = **0x54**) (Category Down = **0x55**)

7.2 Direct Category Select

This section describes how to initiate Direct Category Selection of an XM or Sirius Category (e.g., Hits, Rock, Classical, etc...). Direct Category Selection will select a particular XM or Sirius Category using a discrete command.

NOTE: Once a Category is selected, the first Channel of that Category will be selected.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	4D	00	##	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Direct Category Select = **0x4D**)

Byte #17 = Category HEX Value (XM Category = **See list***) (Sirius Category = **See list***)

*Satellite Radio Category List:

NOTE: While it is not expected that the Category List will change, it is not ruled out by XM or Sirius Satellite Radio.

Value (HEX)	XM Category	Sirius Category
00	None	Pop
01	Decades	Rock
02	Country	Electronica / Dance
03	Hits	Hip-Hop
04	Christian	Country
05	Rock	Jazz / Standards
06	Urban	Classical
07	Jazz & Blues	Latin / International
08	Lifestyle	Sports
09	Dance	Entertainment
0A	Latin	News / Talk
0B	World	Traffic / Weather
0C	Classical	N/A

0D	Kids	N/A
0E	News	N/A
0F	Sports	N/A
10	Comedy	N/A
11	Talk & Entertainment	N/A
12	Special Events	N/A
13	Traffic	N/A

7.3 Category Channel Up / Category Channel Down

Category Channel Up and Category Channel Down will toggle up and down through the channels within the currently selected Satellite Radio Category.

Example: If XM is selected to the Category “Decades” a Category Channel Up message will toggle up only through the Channels that are within the Category “Decades”.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	70	05	02	01	00	02	01	00	##	00	70	00	kk	00	01	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Checksum

Byte #15 = Event ID (Category Channel Up = **0x58**) (Category Channel Down = **0x59**)

8 Display Messages

The following section describes how to read Display Messages. These include Direct Display Feedback messages, Source Broadcast Display Feedback messages, and Multi-Field Broadcast Display Feedback messages. Display messages have a payload which consists of several components: Alignment, Flash Time, the actual Text, the Null Terminating byte, and possibly several arbitrary values that should be ignored. The message can be displayed for a constant amount of time, or a "Flash" display with a specified length of time (Flash Time is in increments of 10ms). The Null Terminating Byte is the Byte consisting of a HEX value **0x00** immediately following the last character of the Text. Any additional Bytes between the Null Terminating Byte and the Checksum are arbitrary values that should be ignored. It is possible that some Display messages will include the special "Invert" character "0xF1" (refer to **section 4.2**)

NOTE: *In order to fully support the display capabilities, Direct Display feedback messages, Source Broadcast Feedback messages and Multi-Field Broadcast Messages, should be supported.*

8.1 Reading Direct Display Feedback

Direct Display Feedback messages are usually sent in direct response to a received command (Example: **AM/FM Tuner = Source 3** with the current frequency being "**102.7 MHz FM**", sending the "**Tune Up**" command as shown in **section 5.2** will trigger the Tuner to send a Display message back to the sender to update the frequency Display to "**102.9 MHz FM**"). The Direct Display Feedback message is sent directly to the Target Device ID of the message sender.

NOTE: *Some of the other bytes within this message may vary. Only the ones necessary to interpret the message are highlighted.*

This is what the Direct Display Feedback message would look like using the above example.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	00	70	00	7D	02	00	02	01	01	02	01	01	00	00	01	00	28	00	01	00	00
Byte #	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46
Value	31	30	32	2E	39	20	4D	48	7A	20	46	4D	00	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A
Byte #	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62							
Value	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	19	F7							

Byte #2 – #4 = Target Device ID (The Device ID of the device that sent the Tune Up command)

Byte #7 = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

Byte #19 = Overall Payload Size

Byte #21 = Alignment (left = **0x01**) (center = **0x00**)

Byte #22 = Flash Time low byte (Flash time is in 10ms increments, **0x00** = Constant)

Byte #23 = Flash Time high byte

Byte #24 – #35 = Text (**102.9 MHz FM**) Refer to ASCII Conversion Chart

Byte #36 = Null Terminating Byte

Byte #61 = Calculated Checksum

8.2 Reading Source Broadcast Display Feedback

This section describes how to read Source Broadcast Display Feedback messages. These Feedback messages are sent to update all devices monitoring a given Source's status. These messages may be sent as a direct result of a sent command or as a general update. The Display Feedback message includes the Source Number indicating which Source the update is intended.

NOTE: *Some of the other bytes within this message may vary. Only the ones necessary to interpret the message are highlighted.*

This example shows a Source Broadcast Display Feedback message with text "**102.9 MHz FM**"

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	7D	00	79	00	7D	02	00	02	01	01	02	01	01	00	00	01	00	28	00	12	00	00
Byte #	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46
Value	31	30	32	2E	39	20	4D	48	7A	20	46	4D	00	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A
Byte #	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62							
Value	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	30	F7							

Byte #4 = Target Keypad ID (**NOTE:** A value of **0x79** in the Target Keypad ID field indicates that this message is a Broadcast Display Feedback message.)

Byte #7 = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

Byte #19 = Overall Payload Size

Byte #21 = Message Type **OR**-ed with the Source Number *(See equation below)

Byte #22 = Flash Time low byte (Flash time is in 10ms increments, **0x00** = Constant)

Byte #23 = Flash Time high byte

Byte #24 – #35 = Text (**102.9 MHz FM**) Refer to ASCII Conversion Chart

Byte #36 = Null Terminating Byte

Byte #61 = Calculated Checksum

*Byte #21 Equation:

In order to calculate the HEX value for **Byte #21** you must take the value for Source Broadcast Display Type (**0x10**) and "**OR**" it with the value of the Source Number that the message applies to (Source 1 = **0x00** ... Source 6 = **0x05**).

Here is the equation for **Byte #21** using the above example as **Source 3**

Source Broadcast Display Type		Source Number		Byte #21
0x10	OR	0x02	=	0x12

8.3 Reading Multi-Field Broadcast Display Feedback Messages

Multi-Field Broadcast Display Feedback messages are only used with Satellite Radio. Multi-Field Broadcast Display Feedback messages are sent to update all devices monitoring a given Source's status and are sent as a direct result of a sent command or as a general update. The Display Feedback message includes the Source Number indicating which Source the update is intended. A "Field ID" is included with the message to indicate which text item is being updated (see table below). Example: Sirius Tuner = **Source 2**, Channel Name = **20:Octane**

Satellite Radio Display “Field ID” list

Field ID (first 6 bits)	Display Field
1	Song Title
2	Artist
3	Genre
6	Channel Number
7	Channel Name

NOTE: Some of the other bytes within this message may vary. Only the ones necessary to interpret the message are highlighted.

This example shows a Multi-Field Broadcast Display Feedback message with text “**20:Octane**”

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	7D	00	79	00	7D	01	00	02	01	01	02	01	01	00	00	01	00	28	00	21	07	1C
Byte #	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46
Value	32	30	3A	4F	63	74	61	6E	65	00	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A
Byte #	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62							
Value	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	5A	09	F7							

Byte #4 = Target Keypad ID (**NOTE:** A value of **0x79** in the Target Keypad ID field indicates that this message is a Broadcast Display Feedback message.)

Byte #7 = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

Byte #19 = Overall Payload Size

Byte #21 = Message Type **OR**-ed with the Source Number * (See equation below)

Byte #22 = Flash Bit and **Field ID** Bits ** (See explanation below)

Bytes #24 – #32 = Text (**20:Octane**) Refer to ASCII Conversion Chart

Byte #33 = Null Terminating Byte

Byte #61 = Calculated Checksum

*Byte #21 Equation:

In order to calculate the HEX value for **Byte #21** you must take the value for Multi-Field Broadcast Display Type (**0x20**) and “**OR**” it with the value of the Source Number that the message applies to (Source 1 = **0x00** ... Source 6 = **0x05**).

Here is the equation for **Byte #21** using the above example as **Source 2**

Multi-Field Display Type		OR	Source Number			Byte #21
0x20			0x01	=		0x21

**Byte #22 Explanation:

Here is a breakdown of the **8-Bits** in **Byte #22**

-**Bits 0 – 5** = Field Id (i.e., **0x07** = **0x47** = **0x87** = **7** = **Channel Name**) Refer to the Field ID list

-**Bits 6 and 7** are to be ignored

-**NOTE:** The special Invert Character (**0xF1**) will be inserted before this Byte if Bit 7 is set

9 Using the Acknowledge Message (Handshaking)

A Peripheral Source such as the Dual Tuner does not require Handshaking when the Keypad ID within the Source Device ID is set to **0x70**. If using a Source Device Keypad ID of **0x70**, and therefore **no** Handshaking, it is best to leave approximately 100ms between messages to ensure that all messages are processed correctly. If Handshaking is desired, it is recommended to use a Source Device Keypad ID of **0x60**. In this case, the Target Device will send an Acknowledge message in response to any High Priority Event messages. Acknowledge messages can be (and normally are) used to trigger the release of any additional messages that may be in the sender's queue.

Example: This is a “Tune Up” Event message sent to a Tuner. Notice that the Event Priority Byte is set to **0x00** (high) and the Source Device Keypad ID is set to **0x60**. This means that the Tuner will send an Acknowledge message in response.

Byte #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	F0	00	7D	kk	00	00	60	05	02	01	00	02	01	00	2F	00	70	00	kk	00	00	xx	F7

kk = Source Number (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Calculated Checksum

Byte #7 = Source Device Keypad ID - **0x60** (indicates that this message requires a Handshake message if the Event Priority is High)

Byte #15 = Event ID (Tune Up = **0x2F**)

Byte #17 = Return Keypad ID - **0x70** (This is the Keypad ID that will receive any resulting Direct Display Feedback message from the Tuner. If this byte is set to **0x70**, it will not require the Source Device to send an Acknowledge message in response to the Direct Display Feedback message. If the original Source Device Keypad ID of **0x60** is used here, you will need to send an Acknowledge message when the Direct Display Feedback message comes back. Failure to send the Acknowledge message will result in a system delay of approximately 2.5 seconds, while the Tuner tries to re-send the data.) **NOTE:** It is recommended to leave this Byte as **0x70**

Byte #21 = Event Priority (High = **0x00**) (Low = **0x01**)

Example: This is an Acknowledge message returned to the Source Device in response to the High Priority Event message of Tune Up as seen above.

Byte #	1	2	3	4	5	6	7	8	9	10	11
Value	F0	00	00	60	00	7D	kk	02	06	xx	F7

kk = Source Number of Event recipient (Source 1 = **0x00** ... Source 6 = **0x05**)

xx = Calculated Checksum

Byte #8 = Message Type (Handshake = **0x02**)

Byte #9 = Handshake Type (Event Handshake = **0x06**) *See Handshake Types

*Handshake Types

Value (HEX)	Handshake Type
0x02	Data Handshake
0x06	Event Handshake

10 Command Code Summary for Peripheral Tuner

Tuner Command Event IDs	HEX Value (Byte 15)	Document Sections
Common Commands		
DIGIT #0	0A	5.3
DIGIT #1	01	5.3
DIGIT #2	02	5.3
DIGIT #3	03	5.3
DIGIT #4	04	5.3
DIGIT #5	05	5.3
DIGIT #6	06	5.3
DIGIT #7	07	5.3
DIGIT #8	08	5.3
DIGIT #9	09	5.3
PRESET UP	0E	5.5
PRESET DOWN	0F	5.5
DIRECT BANK SELECTION MODE	15	5.8
DIRECT PRESET SELECTION MODE	16	5.6
DIRECT TUNING MODE (FREQ or CHAN)	17	5.3
BANK UP	29	5.7
BANK DOWN	2A	5.7
TUNE UP (FREQUENCY or CHANNEL)	2F	5.2
TUNE DOWN (FREQUENCY or CHANNEL)	30	5.2
POWER ON	3A	5.1
POWER OFF	3B	5.1
PRESET SAVE	3C - 41	5.4
AM/FM Commands		
SEEK	1B	6.1
SCAN	46	6.2
AM FM TOGGLE	1A	6.3
FM SELECT	49	6.4
AM SELECT	4A	6.4
STEREO / MONO TOGGLE	13	6.5
DIRECT STEREO SELECT	33	6.6
DIRECT MONO SELECT	34	6.6
LOCAL / DISTANT TOGGLE	14	6.7
DIRECT LOCAL SELECT	42	6.8

DIRECT DISTANT SELECT	43	6.8
Satellite Radio Commands (XM and Sirius)		
CATEGORY SELECT	4D	7.2
CATEGORY UP	54	7.1
CATEGORY DOWN	55	7.1
CAT CHANNEL UP	58	7.3
CAT CHANNEL DOWN	59	7.3

11 ASCII Character Set to HEX Conversion Chart

ASCII	=	HEX	ASCII	=	HEX	ASCII	=	HEX
Space		20	@		40	`		60
!		21	A		41	a		61
"		22	B		42	b		62
#		23	C		43	c		63
\$		24	D		44	d		64
%		25	E		45	e		65
&		26	F		46	f		66
'		27	G		47	g		67
(		28	H		48	h		68
)		29	I		49	i		69
*		2A	J		4A	j		6A
+		2B	K		4B	k		6B
,		2C	L		4C	l		6C
-		2D	M		4D	m		6D
.		2E	N		4E	n		6E
/		2F	O		4F	o		6F
0		30	P		50	p		70
1		31	Q		51	q		71
2		32	R		52	r		72
3		33	S		53	s		73
4		34	T		54	t		74
5		35	U		55	u		75
6		36	V		56	v		76
7		37	W		57	w		77
8		38	X		58	x		78
9		39	Y		59	y		79
:		3A	Z		5A	z		7A
;		3B	[		5B	{		7B
<		3C	\		5C			7C
=		3D	]		5D	}		7D
>		3E	^		5E	~		7E
?		3F	_		5F	Del		7F

JW 12/14/05